Market Basket Analysis

Finding frequent itemsets in the IMDB Movies Dataset

Greta Zehnder

June 2026

Contents

1 Introduction

Market basket analysis is a data mining technique aimed at discovering associations among items that frequently co-occur in transactions. Originally developed in the context of retail, where the goal is to identify products that customers tend to purchase together, the approach generalizes naturally to any domain where data can be represented as a collection of sets.

In this project, the technique is applied to a movie dataset, where each film is modeled as a basket and its main actors as items. The goal is to discover frequent co-acting patterns, i.e., groups of actors who appear together in multiple films. The analysis is carried out through a custom implementation of the Apriori algorithm, which is applied to a filtered version of the dataset and evaluated both in terms of the itemsets it produces and its runtime behavior as the dataset size increases.

The remainder of the report is organized as follows. Section 2 describes the dataset and the preprocessing pipeline. Section 3 covers the exploratory analysis and the choice of the minimum support threshold. Section 4 presents the Apriori implementation in detail. Section 5 discusses the frequent itemsets found, and Section 6 reports the scalability experiment.

2 Dataset and preprocessing

2.1 Dataset

The dataset used in this project is the *IMDB Top 1000 Movies and TV Shows Dataset* [1], publicly available on Kaggle under the Public Domain license. It contains information about the top 1000 movies and TV shows ranked on IMDB, organized into 16 columns that include metadata such as title, release year, genre, IMDB rating, director, and gross revenue, as well as four actor fields (`Star1`, `Star2`, `Star3`, `Star4`) listing the main cast members of each title.

For the purposes of this project, only the four actor columns are relevant. Each row is treated as a transaction (basket), and the actors listed in `Star1`–`Star4` are treated as items, so that the dataset is naturally mapped to the market basket setting without requiring any structural transformation beyond column selection.

2.2 Preprocessing

The preprocessing pipeline starts with column selection, retaining only the four actor fields (`Star1`, `Star2`, `Star3`, `Star4`). Rows with at least one missing value are then dropped using `dropna()`: after this step, all 1000 rows are preserved, meaning that no missing values were present in the actor columns. Each row is subsequently converted into a list of four actors, resulting in 1000 transactions that constitute the input to the subsequent analysis steps.

The support-based filtering applied before running Apriori, which reduces the number of distinct items and thereby limits both memory usage and computation time, is described in Section 3, as it is motivated by the frequency analysis carried out in that phase.

3 Exploratory analysis and support threshold selection

3.1 Actor frequency analysis

Before running the mining algorithm, the frequency distribution of actors across all transactions is analyzed, with three objectives in mind: determining the total number of distinct items, identifying the most frequently appearing actors, and informing the choice of a minimum support threshold that is both meaningful and computationally feasible.

The dataset contains 2709 distinct actors. Table 1 shows the top 10 most frequent actors, with Robert De Niro appearing in 17 films, Tom Hanks in 14, and Al Pacino in 13.

Actor	**Appearances**
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 1: Top 10 most frequent actors in the dataset.

3.2 Support threshold selection

A minimum support threshold of 1% (at least 10 appearances out of 1000 films) was initially considered. However, this value proved too restrictive, leaving only 9 actors above the threshold and severely limiting the discovery of meaningful co-occurrence patterns.

The threshold was therefore lowered to 0.5% (at least 5 appearances), a level at which 79 actors are retained. Transactions containing none of these actors are also dropped, reducing the dataset to 399 baskets. This filtered dataset provides a richer set of itemsets to mine while keeping the one-hot representation tractable: rather than working with a 1000×2709 binary matrix, the algorithm operates on a much more compact 399×79 one.

The minimum count used by Apriori is set to 5, the same threshold used to filter actors, computed as $\lfloor 0.005 \times 1000 \rfloor = 5$ on the original 1000 films. This keeps the definition of frequent consistent throughout the pipeline: an item or itemset is frequent if it appears in at least 5 of the 1000 original films. Support values are therefore computed as ratios over the original 1000 transactions, so that they reflect the frequency of a pattern within the full dataset rather than within the filtered subset.

4 Frequent itemset mining with Apriori

Apriori is the foundational algorithm for frequent itemset mining, and it is based on the monotonicity property: if an itemset is infrequent, all of its supersets are infrequent as

well. This property allows the candidate space to be pruned level by level, so that only itemsets whose subsets have already been found to be frequent are counted, avoiding the need to enumerate all possible combinations.

The algorithm proceeds in passes over the dataset. In the first pass, the support of all individual items is counted and only those meeting the minimum support threshold are retained, forming the set of frequent 1-itemsets L_1. In each subsequent pass k, candidate k-itemsets are generated by merging pairs of frequent $(k-1)$-itemsets and pruning those whose $(k-1)$-subsets are not all frequent, their support is then counted by scanning the baskets, and only those reaching the threshold are kept. The process continues until no new frequent itemsets are produced.

The implementation is organized around three helper functions and a main driver. `count_singletons(baskets)` performs the first pass by scanning all baskets and returning a `Counter` with the support count of each individual item. `generate_candidates(prev_frequent, k)` takes the list of frequent $(k-1)$-itemsets and generates candidate k-itemsets by merging pairs: each candidate is kept only if all its $(k-1)$-subsets are present in the set of previously found frequent itemsets, which enforces monotonicity. `count_candidates(baskets, candidates, frequent_items, k)` scans the baskets and counts how many of them contain each candidate, restricting attention to items that are already known to be frequent singletons within each basket, again by monotonicity.

The main `apriori(baskets, min_count)` function calls these three helpers in sequence for $k = 1, 2, 3$, stops if no candidates are generated at a given level, and returns a dictionary mapping each value of k to the corresponding set of frequent itemsets together with their counts. In this project, the function is run on the filtered dataset (399 baskets, 79 actors) with min_count = 5.

5 Results

The algorithm identifies 79 frequent 1-itemsets, 3 frequent pairs, and 1 frequent triple. Table 2 shows the pairs sorted by support, where support is computed as the ratio between the count and the total number of original films (1000).

Actor pair	**Support**	**Count**
Daniel Radcliffe, Rupert Grint	0.0060	6
Daniel Radcliffe, Emma Watson	0.0050	5
Emma Watson, Rupert Grint	0.0050	5

Table 2: Frequent actor pairs found by Apriori.

The most frequent co-acting pattern is (Daniel Radcliffe, Rupert Grint) with a count of 6, reflecting their repeated collaboration in the Harry Potter series. All three pairs involve Harry Potter cast members, which is consistent with the franchise's large number of entries in the IMDB Top 1000.

Moving to 3-itemsets, Table 3 reports the single frequent triple identified by the algorithm: the full Harry Potter core trio, which appears together in 5 films.

Actor triple	Support	Count
Daniel Radcliffe, Emma Watson, Rupert Grint	0.0050	5

Table 3: Frequent 3-itemset found by Apriori.

It is worth noting that, at this support level, the patterns found are entirely driven by the Harry Potter franchise, where the same core cast recurs across multiple entries. This is a natural consequence of the dataset being composed of the most popular titles, where franchise entries tend to be well represented, combined with the relatively high minimum support threshold of 5 appearances.

6 Scalability experiment

6.1 Experimental setup

To evaluate how runtime scales with dataset size, increasing fractions of the filtered transactions are sampled (30%, 60%, 100%) and the average runtime over 10 independent runs is measured using `time.perf_counter()`, which provides sub-millisecond resolution and is more precise than the standard `time.time()` function. Averaging over multiple runs is motivated by the need to reduce the effect of random fluctuations in execution time caused by system-level factors such as CPU scheduling and memory caching. A fixed random seed (42) is used to ensure that the same subsamples are drawn across experiments, and the minimum count threshold is kept fixed at 5 across all fractions.

6.2 Results

Table 4 reports the average runtime for each dataset size, and Figure 1 shows the corresponding growth curve.

Fraction	Baskets	Avg runtime (s)
0.3	119	0.0010
0.6	239	0.0042
1.0	399	0.0344

Table 4: Apriori average runtime as dataset size increases (10 runs each).

6.3 Discussion

The runtime growth observed in this experiment is superlinear: going from 119 to 399 baskets (a factor of approximately 3.4) corresponds to a runtime increase from 0.0010s to 0.0344s (a factor of approximately 34). This behavior is explained by the interaction between the dataset size and the support threshold. At smaller fractions, very few actor combinations reach the minimum count of 5, so the candidate counting step has little work to do. As the dataset grows, more candidates accumulate enough support to be counted, increasing the workload in a non-linear fashion. This is a known characteristic of frequent itemset mining at the boundary of the support threshold: when the dataset

is too small relative to the threshold, almost nothing is frequent and the algorithm is artificially fast.

It should be noted that this experiment uses real subsamples of the filtered dataset, which preserves the original data distribution, rather than synthetically replicating transactions. This approach provides a more realistic picture of scalability behavior, although the dataset remains small and the absolute runtimes are correspondingly low. On a real larger dataset with more distinct actors and sparser co-occurrences, the candidate generation step would be more expensive and the runtime growth could be more pronounced.

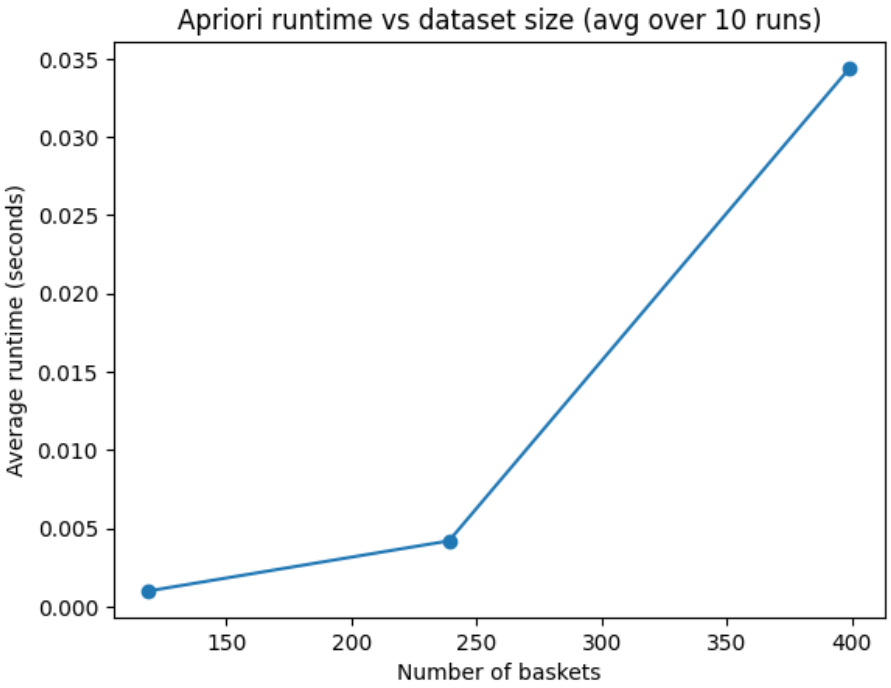

Figure 1: Apriori runtime vs dataset size (average over 10 runs).

7 Conclusions

This project applied market basket analysis to the IMDB Top 1000 Movies and TV Shows Dataset, modeling movies as transactions and actors as items. After preprocessing and support-based filtering (threshold: 0.5%), the working dataset was reduced to 399 baskets and 79 distinct actors. A custom implementation of Apriori, organized around three helper functions that enforce monotonicity at each step, was then run on this filtered dataset with a minimum count of 5, consistent with the filtering threshold applied to the original 1000 films.

The algorithm identified 79 frequent singletons, 3 frequent pairs, and 1 frequent triple. All patterns involve Harry Potter cast members, a result consistent with the high minimum support threshold applied (as previously mentioned).

The scalability experiment showed that runtime grows superlinearly with the number of baskets, an effect explained by the interaction between dataset size and the support threshold: at smaller fractions few candidates reach the minimum count and the algorithm is artificially fast, while as the dataset grows more candidates accumulate sufficient

support and the workload increases non-linearly. A limitation of this evaluation is that it operates on a relatively small dataset, so the observed trends may not fully capture the algorithm's behavior at larger scales.

Future work could extend the analysis to a larger and more diverse movie dataset, which would provide a more stress-test of scalability and potentially reveal co-acting patterns that span across productions rather than being driven by franchise recurrence. Association rule mining on top of the frequent itemsets, extracting directional patterns of the form "if actor A appears, actor B is likely to appear as well", would also be a natural extension of the current pipeline.

References

[1] Harshit Shankhdhar, *IMDB Movies Dataset*, Kaggle. https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows

Declaration of Academic Integrity

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.